

Teach Module 4: Java Memory Management and Performance

This guide teaches Module 4 independently from the course material. The course document was used only to identify the module topic: **Memory Management and Performance**.

Learning Goals

By the end of this module, you should be able to explain:

- The difference between stack and heap memory
- How Java object references work
- When objects become eligible for garbage collection
- Why Java applications can still have memory leaks
- What `StackOverflowError` and `OutOfMemoryError` mean
- How basic performance choices affect memory and speed
- How to observe memory usage during development

1. Stack vs Heap

Java memory can be understood through two important areas: the **stack** and the **heap**.

Stack

The stack stores method calls and local variables.

```
public void greet() {  
    int age = 25;  
    String name = "Asha";  
}
```

When `greet()` runs, Java creates a stack frame for that method.

- `age` is stored directly in the stack frame.
- `name` is a reference variable.
- The actual `String` object lives on the heap.

When the method finishes, its stack frame disappears.

The stack is fast, organized, and short-lived.

Heap

The heap stores objects created at runtime.

```
Customer customer = new Customer("Asha");
```

Here:

- `customer` is a reference variable.
- The actual `Customer` object lives on the heap.

Objects on the heap remain alive as long as they can still be reached through a live reference.

```
Customer customer = new Customer("Asha");
customer = null;
```

After `customer = null`, if no other reference points to that object, it becomes eligible for garbage collection.

Key idea:

Stack memory is tied to method execution. Heap memory is tied to object lifetime.

2. Object Lifecycle

A Java object usually goes through these stages:

1. Created

```
Order order = new Order();
```

2. Used

```
order.setAmount(100);
```

3. Becomes unreachable

```
order = null;
```

4. Garbage collected later by the JVM

You do not control exactly when garbage collection happens. You control whether your code keeps unnecessary references.

Example of memory pressure:

```
List<byte[]> data = new ArrayList<>();

while (true) {
    data.add(new byte[1024 * 1024]);
}
```

Each loop adds a new 1 MB array. Because the list keeps references to all arrays, garbage collection cannot remove them.

3. Garbage Collection

Garbage collection, or GC, is the JVM process that reclaims heap memory from unreachable objects.

An object is collectible when it can no longer be reached from active code.

```
public void process() {  
    Invoice invoice = new Invoice();  
}
```

After `process()` finishes, the local variable `invoice` disappears. If nothing else references the `Invoice`, it can be collected.

Garbage collection helps prevent many memory problems, but it does not prevent all memory leaks.

A Java memory leak usually means:

An object is no longer needed, but the program still holds a reference to it.

Example:

```
private static final List<User> users = new ArrayList<>();  
  
public void addUser(User user) {  
    users.add(user);  
}
```

If users are added forever and never removed, memory keeps growing.

4. Generational Garbage Collection

Many Java garbage collectors are based on this observation:

Most objects die young.

Temporary objects are created constantly.

```
String result = firstName + " " + lastName;
```

Many short-lived objects disappear quickly. To optimize for that, the JVM often divides heap memory into areas such as:

- Young generation: new objects
- Old generation: objects that have survived long enough
- Metadata areas: JVM/runtime class information, depending on JVM version

Young-generation collection is usually cheaper than scanning the entire heap.

5. G1 GC and ZGC

G1 GC

G1 stands for Garbage First. It is a common garbage collector for modern Java server applications.

G1 divides the heap into regions and tries to collect regions that provide the most benefit first. It balances application throughput with predictable pause times.

G1 is a strong default choice for many enterprise Java services.

ZGC

ZGC is designed for very low pause times, even with large heaps.

It performs much of its work while the application continues running. This makes it useful for systems where long application pauses are unacceptable.

Simple comparison:

Collector	Good For
G1	General server applications
ZGC	Very low-latency applications with large heaps

6. Performance Basics

Performance is not only about making code faster. In Java, performance can involve:

- CPU usage
- Memory usage
- Garbage collection behavior
- Database or API latency
- Object allocation rate
- Thread behavior

A beginner mistake is optimizing before measuring.

Better process:

1. Measure first.
2. Find the actual bottleneck.
3. Change one thing.
4. Measure again.

7. Common Java Performance Problems

Creating unnecessary objects

```
for (int i = 0; i < 1_000_000; i++) {  
    String message = new String("hello");  
}
```

Better:

```
String message = "hello";  
  
for (int i = 0; i < 1_000_000; i++) {  
    // Reuse message  
}
```

Keeping references too long

```
Map<String, UserSession> sessions = new HashMap<>();
```

If old sessions are never removed, memory grows.

Loading too much data at once

```
List<Customer> customers = customerRepository.findAll();
```

If the table is huge, this can cause memory and performance problems. Pagination is usually better.

String concatenation in loops

```
String result = "";

for (String item : items) {
    result += item;
}
```

Better:

```
StringBuilder result = new StringBuilder();

for (String item : items) {
    result.append(item);
}
```

8. Diagnosing Memory Issues

Common symptoms:

- Application becomes slower over time
- High CPU usage during garbage collection
- `OutOfMemoryError`
- Long pauses
- Heap usage increases and does not drop

Common errors:

```
java.lang.OutOfMemoryError: Java heap space
```

This means the heap ran out of space.

```
java.lang.StackOverflowError
```

This usually means too many nested method calls, often caused by recursion without a proper stopping condition.

Example:

```
public void callForever() {  
    callForever();  
}
```

Mental Model

Remember:

The stack answers: what method is running right now?

The heap answers: what objects exist right now?

For garbage collection:

If an object cannot be reached, it can be cleaned.

If your code still references it, the JVM must keep it.

Practice Exercises

Exercise 1: Stack vs Heap

Write a small program with a method that creates primitive variables and objects.

```
public class MemoryDemo {  
    public static void main(String[] args) {  
        int number = 10;  
        Student student = new Student("Asha");  
        printStudent(student);  
    }  
  
    static void printStudent(Student student) {  
        System.out.println(student.getName());  
    }  
}
```

Explain which parts are on the stack and which parts are on the heap.

Exercise 2: Object References

Create two references pointing to the same object.

```
Student s1 = new Student("Ravi");  
Student s2 = s1;  
  
s1 = null;  
  
System.out.println(s2.getName());
```

Explain why the object is still alive after `s1 = null` .

Exercise 3: Garbage Collection Eligibility

Write a method that creates an object and then returns.

```
static void createObjects() {  
    Student s = new Student("Maya");  
}
```

After the method finishes, identify which objects are eligible for garbage collection.

Exercise 4: Memory Leak Simulation

Create a static list and keep adding objects to it.

```
static List<byte[]> memory = new ArrayList<>();  
  
public static void main(String[] args) {  
    while (true) {  
        memory.add(new byte[1024 * 1024]);  
    }  
}
```

Run it with a small heap:

```
java -Xmx64m MemoryLeakDemo
```

Observe the `OutOfMemoryError`.

Exercise 5: Fix the Memory Leak

Modify the previous exercise so old data is removed.

```
if (memory.size() > 10) {  
    memory.remove(0);  
}
```

Compare the behavior before and after.

Exercise 6: StackOverflowError

Create a recursive method without a stopping condition.

```
static void recurse() {  
    recurse();  
}
```

Then fix it:

```
static void recurse(int count) {  
    if (count == 0) {
```

```
        return;
    }
    recurse(count - 1);
}
```

Exercise 7: String Concatenation Performance

Compare string concatenation with `StringBuilder` .

```
String result = "";
for (int i = 0; i < 100000; i++) {
    result += i;
}
```

Then rewrite:

```
StringBuilder builder = new StringBuilder();
for (int i = 0; i < 100000; i++) {
    builder.append(i);
}
```

Measure execution time using `System.currentTimeMillis()` .

Exercise 8: Object Creation in Loops

Create many temporary objects in a loop and measure memory usage.

```
for (int i = 0; i < 1_000_000; i++) {
    Student s = new Student("Student " + i);
}
```

Compare this with a version that reuses objects where reasonable.

Exercise 9: Runtime Memory Info

Print heap memory usage before and after creating objects.

```
Runtime runtime = Runtime.getRuntime();

System.out.println("Free memory: " + runtime.freeMemory());
System.out.println("Total memory: " + runtime.totalMemory());
System.out.println("Max memory: " + runtime.maxMemory());
```

Exercise 10: GC Observation

Run a program with GC logging enabled.

```
java -Xlog:gc MemoryDemo
```

Create many objects and watch when garbage collection happens.

Lab: Java Memory and Garbage Collection

Goal

Understand stack vs heap, object references, garbage collection eligibility, memory leaks, and basic performance measurement.

Lab Setup

Create a file named:

```
MemoryManagementLab.java
```

Part 1: Stack vs Heap

```
class Student {
    private String name;

    Student(String name) {
        this.name = name;
    }

    String getName() {
        return name;
    }
}

public class MemoryManagementLab {
    public static void main(String[] args) {
        int score = 95;
        Student student = new Student("Asha");

        printStudent(student, score);
    }

    static void printStudent(Student student, int score) {
        System.out.println(student.getName() + " scored " + score);
    }
}
```

Run:

```
javac MemoryManagementLab.java
java MemoryManagementLab
```

Answer:

1. Where is `score` stored?
2. Where is the `Student` object stored?
3. What is passed into `printStudent` : the object or a reference?

Part 2: Object References

Add this method:

```
static void referenceDemo() {  
    Student s1 = new Student("Ravi");  
    Student s2 = s1;  
  
    s1 = null;  
  
    System.out.println(s2.getName());  
}
```

Call it from `main` .

Answer:

1. Is the `Student` object garbage collected after `s1 = null` ?
2. Why does `s2.getName()` still work?

Part 3: Garbage Collection Eligibility

Add:

```
static void gcEligibilityDemo() {  
    Student student = new Student("Maya");  
    System.out.println(student.getName());  
}
```

Call it from `main` .

Answer:

1. When does the `Student("Maya")` object become eligible for garbage collection?
2. Does Java immediately delete it? Why or why not?

Part 4: Stack Overflow

Add:

```
static void stackOverflowDemo() {  
    stackOverflowDemo();  
}
```

Do not call it immediately.

Then call it only when you are ready:

```
// stackOverflowDemo();
```

Uncomment and run it.

Expected result:

```
java.lang.StackOverflowError
```

Then comment it again so the rest of the lab can run.

Part 5: Memory Leak Simulation

Add imports at the top:

```
import java.util.ArrayList;
import java.util.List;
```

Add this field:

```
static List<byte[]> memoryLeak = new ArrayList<>();
```

Add this method:

```
static void memoryLeakDemo() {
    int count = 0;

    while (true) {
        memoryLeak.add(new byte[1024 * 1024]);
        count++;
        System.out.println("Added " + count + " MB");
    }
}
```

Call it from `main` only when ready.

Run with a small heap:

```
java -Xmx64m MemoryManagementLab
```

Expected result:

```
java.lang.OutOfMemoryError: Java heap space
```

Part 6: Fix the Leak

Replace the loop body with:

```
memoryLeak.add(new byte[1024 * 1024]);

if (memoryLeak.size() > 10) {
    memoryLeak.remove(0);
}
```

Now the program should keep running longer because old objects are no longer permanently retained.

Part 7: Measure Performance

Add:

```
static void stringPerformanceDemo() {
    long start = System.currentTimeMillis();

    String result = "";
    for (int i = 0; i < 100_000; i++) {
        result += i;
    }

    long end = System.currentTimeMillis();
    System.out.println("String concat time: " + (end - start) + " ms");

    start = System.currentTimeMillis();

    StringBuilder builder = new StringBuilder();
    for (int i = 0; i < 100_000; i++) {
        builder.append(i);
    }

    end = System.currentTimeMillis();
    System.out.println("StringBuilder time: " + (end - start) + " ms");
}
```

Call it from `main` .

Observe which version is faster.

Part 8: Print Memory Usage

Add:

```
static void printMemory() {
    Runtime runtime = Runtime.getRuntime();

    long free = runtime.freeMemory();
    long total = runtime.totalMemory();
    long max = runtime.maxMemory();

    System.out.println("Free memory: " + free);
    System.out.println("Total memory: " + total);
    System.out.println("Max memory: " + max);
}
```

Call it before and after creating objects.

Final Lab Questions

1. Why can Java still have memory leaks even though it has garbage collection?

2. What causes `StackOverflowError` ?
3. What causes `OutOfMemoryError` ?
4. Why is `StringBuilder` better inside loops?
5. What does `-Xmx64m` control?

Challenge

Create an `OrderProcessor` that processes 1,000 orders in batches of 100.

Requirements:

- Create an `Order` class.
- Store orders in a list.
- Process 100 at a time.
- Remove processed orders.
- Print memory usage after each batch.
- Avoid keeping old references unnecessarily.

Quick Check

Before moving to Module 5, make sure you can answer:

1. What is stored on the stack?
2. What is stored on the heap?
3. When does an object become eligible for garbage collection?
4. Why can Java still have memory leaks?
5. What is the difference between `OutOfMemoryError` and `StackOverflowError` ?